自动化测试程序快速上手说明

1. 项目概述

这是一个基于 **Python + Ginkgo SPI** 适配器 **+ E5052B** 的自动化测试工程，主要用于：

- 通过 **SPI** 对芯片寄存器/字段进行读写
- 按寄存器字段名而不是裸地址进行操作
- 对单个字段或多个字段做遍历扫描（sweep）
- 在 SPI 操作后联动 **E5052B** 进行相噪抓取
- 自动导出截图、CSV、Excel 和汇总结果
- 通过脚本化 case 组织完整测试流程（full-flow）

从工程结构上看，这个项目已经不是“单一脚本”，而是一个分层较清晰的测试框架，适合后续继续扩展新的芯片配置、寄存器表、测试 case 和仪器联动能力。

---

2. 这个工程能做什么

2.1 SPI 基础能力

- 指定寄存器地址写入数据
- 指定寄存器地址读回数据
- 写后立即回读验证
- 对寄存器地址范围做 sweep

2.2 字段级操作能力

- 根据 字段名 直接读写，而不是手动算 bit 位
- 支持字段别名解析
- 支持字段写入后自动验证
- 支持字段 sweep
- 支持 shadow 模式构造寄存器值

2.3 全流程测试能力

- 先做 SPI 配置
- 再调用 E5052B 抓取相噪
- 可选等待时间、截图、CSV、Excel
- 自动生成 run 目录和输出文件
- 支持单 case 输出结果归档

---

3. 当前确认的协议与默认参数

根据工程中的 README.md、main.py 和配置文件，当前默认使用的是以下 SPI 工作方式：

3.1 协议帧格式

- 写地址帧：00 [addr]
- 写数据帧：40 [data]
- 读帧：80 [addr]
- 回读字节索引：raw_rx[1]

3.2 当前默认 SPI 参数

- control_mode = 0
- cpol = 0
- cpha = 0
- clock_hz = 500000

也就是你当前工程默认按 **500 kHz**、**Mode 0** 风格运行。

---

4. 推荐给使用者先看的目录

对于第一次接手这个工程的人，建议按下面顺序阅读：

1. README.md
2. main.py
3. config/active/
4. protocol/
5. scripts/basic/
6. scripts/fullflow/
7. instruments/e5052b/
8. outputs/

这样可以最快建立“配置 → 协议 → 命令 → case → 输出结果”的完整认知。

---

5. 文件架构说明

下面是按功能整理后的项目结构说明。

```
Auto_test_proj10/
├─ main.py                  # 主命令入口，支持寄存器/字段读写、验证、sweep
├─ README.md                    # 工程概览与基础示例
```

```
├─ requirements.txt            # 依赖（当前看到 xlsxwriter、pillow）
├─ FRAMEWORK.md                # 简短框架说明入口
│
├─ config/                     # 配置层
│  ├─ active/                  # 当前激活配置
│  │  ├─ project_config.json   # 项目主配置
│  │  ├─ registers.json        # 寄存器地址表
│  │  ├─ register_fields.json  # 寄存器字段表
│  │  └─ spi_config.json       # 当前 SPI 配置
│  ├─ chip_default/            # 默认芯片配置模板
│  ├─ common/                  # 公共配置，如 runtime policy、field alias
│  └─ instrument/
│     └─ e5052b_config.json    # E5052B 连接与导出配置
│
├─ driver/                     # 设备驱动封装层
│  └─ control_spi.py           # Ginkgo DLL 调用与 SPI 设备控制
│
├─ protocol/                   # 协议层
│  ├─ chip_spi.py              # 芯片 SPI 协议读写主类
│  └─ register_map.py          # 字段/别名/位段解析
│
├─ core/                       # 测试流程通用层
│  ├─ formatter.py             # 终端输出格式化
│  ├─ measurement_hook.py      # 测量挂钩逻辑
│  ├─ result_model.py          # 结果模型
│  ├─ session.py               # 会话与流程组织
│  ├─ sweeper.py               # preset/sweep 相关逻辑
│  └─ verifier.py              # 校验逻辑
│
├─ instruments/                # 仪器集成层
│  ├─ inst_5052b.py            # E5052B 封装入口
│  ├─ instrument_base.py       # 仪器基类
│  └─ e5052b/
│     ├─ client.py             # SCPI 客户端
│     ├─ capture.py            # 单次抓取逻辑
│     ├─ exporter.py           # 结果导出
│     ├─ metrics.py            # 指标提取
│     └─ models.py             # 数据模型
│
├─ scripts/                    # 日常执行脚本
```

```
│   ├── basic/                  # 纯 SPI 基础示例
│   ├── loops/                  # 纯 SPI sweep 示例
│   ├── fullflow/               # SPI + E5052B 联动 case
│   └── templates/              # 新 case 编写模板
│
├── tests/                      # 基础测试与验证脚本
│
├── utils/                      # 通用工具
│   ├── e5052b_config.py        # 读取 E5052B 配置
│   ├── runtime_policy.py       # 运行策略加载与覆盖
│   ├── value_spec.py           # 离散/连续值规格处理
│   └── 其他 path/json/file 工具
│
├── lib/windows/64bit/
│   └── Ginkgo_Driver.dll       # Ginkgo 官方驱动 DLL
│
├── outputs/                    # 测试输出目录
│   ├── e5052b/                 # 仪器单独抓图输出
│   └── fullflow/               # 全流程 case 输出
│
└── logs/                       # 日志目录
```

---

6. 各层职责说明

6.1 main.py：命令行入口

这是最适合新用户直接上手的入口。它封装了最常见的动作：

- write
- read
- write-read
- sweep
- field-list
- field-read
- field-write
- field-write-read
- field-sweep

也就是说，日常做寄存器/字段调试时，优先用 **main.py**，不需要一开始就进 full-flow 脚本。

6.2 driver/：底层硬件访问

这一层负责通过 ctypes 调用 Ginkgo_Driver.dll，完成：

- 扫描设备
- 打开设备
- 初始化 SPI
- 写 SPI
- 写读 SPI
- 关闭设备

如果后续出现“设备找不到”“初始化失败”“DLL 加载异常”等问题，优先检查这一层。

6.3 protocol/：芯片协议与寄存器抽象

这一层是整个项目最核心的抽象层。

- ChipSpi：封装了芯片读写协议
- RegisterMap：把字段 JSON 转为可读写对象

使用者可以不用再手算掩码和 bit 偏移，而是直接：

- 按寄存器地址操作
- 按字段名操作
- 按字段 alias 操作

6.4 scripts/fullflow/：完整测试 case

这一层负责把：

- SPI 操作
- preset 设置
- sweep 遍历
- 等待时间
- E5052B 抓图
- 输出整理

串成完整的测试流程。

所以如果目标是“正式测试、批量跑 case、联动仪器采数据”，主要看这里。

6.5 instruments/e5052b/：相噪仪集成

这一层把 E5052B 封装成独立模块，避免把仪器逻辑写死在主 SPI 框架里。

优点是：

- SPI 和仪器逻辑解耦
- 可单独验证 E5052B 能否工作
- 可在不同 full-flow case 中复用
- 后续扩展别的仪器时结构也比较统一

---

7. 配置文件说明

7.1 config/active/project_config.json

主要定义当前项目用哪一组配置文件，例如：

- 寄存器字段文件路径
- 寄存器地址文件路径
- SPI 配置文件路径
- 输出目录
- 日志目录

它相当于“当前工程激活哪套芯片配置”的总入口。

7.2 config/active/spi_config.json

这里是当前实际使用的 SPI 参数，已看到的默认值包括：

- device_index = 0
- spi_index = 0
- control_mode = 0
- cpol = 0
- cpha = 0
- clock_hz = 500000
- use_shadow = false

当硬件环境、接线方式或测试目标变化时，这个文件通常是首要调整点之一。

7.3 config/active/register_fields.json

这是字段级控制的关键配置，里面定义了：

- 字段名
- alias
- 地址
- bit 范围

- 默认值
- 最大值

只要这份表定义准确，就可以在命令行直接使用字段名读写。

7.4 config/instrument/e5052b_config.json

这里管理 E5052B 的相关参数，例如：

- IP 地址
- 端口
- 超时时间
- 默认输出目录
- 是否保存截图
- 是否保存 CSV
- 是否保存 Excel
- 抓取前等待时间
- 是否 freeze/resume
- 关键 offset 点

当前配置中可见仪器地址是：

- host = 192.168.18.58
- port = 5025

如果换了实验室网络环境或仪器 IP，优先修改这里。

---

8. 环境准备

8.1 基本依赖

当前工程里明确看到的 Python 依赖有：

pip install xlsxwriter pillow

如果后续出现缺少模块报错，再按实际报错补装即可。

8.2 硬件准备

运行本工程前，一般需要确认：

- Ginkgo SPI 适配器已连接电脑
- 芯片板卡已正确连接 SPI 信号

- 电源上电正常
- E5052B 已联网且可通过 IP 访问
- Windows 能正常加载 Ginkgo_Driver.dll

8.3 运行目录

建议始终在 工程根目录 执行命令，否则相对路径可能找不到，例如：

- config/...
- outputs/...
- lib/windows/64bit/Ginkgo_Driver.dll

---

9. 最快上手流程

对于新用户，建议按下面顺序验证。

第一步：列出所有字段

python main.py field-list

用途：

- 确认字段表加载正常
- 查看当前可操作的字段名和 alias
- 避免后面拼错字段名

第二步：读取某个字段

python main.py field-read --name CFG_OSC2_GMP_NUM

用途：

- 确认 SPI 通信通了
- 确认字段映射正常
- 看看终端读回格式是否符合预期

第三步：写入并回读验证

python main.py field-write-read --name CFG_OSC2_GMP_NUM --value 5

用途：

- 完成最基础的数据闭环
- 终端会显示 PASS / FAIL

第四步：做字段 **sweep**

python main.py field-sweep --name CFG_OSC2_GMP_NUM --start 0 --stop 7 --step 1

用途：

- 快速确认整个字段取值范围能否稳定写读

第五步：单独验证 E5052B 抓取

```
python  -m scripts.fullflow.e5052b_capture_once
```

用途：

- 确认相噪仪网络联通
- 确认截图/CSV/Excel 导出链路正常

第六步：验证完整联动 case

例如：

```
python  -m scripts.fullflow.case_spi_then_capture
```

用途：

- 先执行 SPI，再执行 E5052B 抓取
- 看输出目录是否完整生成

---

10. 常用命令速查

10.1 寄存器级命令

写寄存器：

```
python main.py write --reg 0x20 --value 0x7c
```

读寄存器：

```
python main.py read --reg 0x20
```

写后回读验证：

```
python main.py write-read --reg 0x20 --value 0x7c
```

对寄存器做 sweep：

```
python main.py sweep --reg 0x20 --start 0x00 --stop 0x07 --step 0x01
```

10.2 字段级命令

列字段：

```
python main.py field-list
```

读字段：

python main.py field-read --name CFG_OSC2_GMP_NUM

写字段：

python main.py field-write --name CFG_OSC2_GMP_NUM --value 5

写字段并验证：

python main.py field-write-read --name CFG_OSC2_GMP_NUM --value 5

字段 sweep：

python main.py field-sweep --name CFG_OSC2_GMP_NUM --start 0 --stop 7 --step 1

使用 shadow 模式：

python main.py field-write-read --name CFG_OSC2_GMP_NUM --value 5 --use-shadow

---

11. Full-flow 脚本说明

scripts/fullflow/ 是整个工程最重要的"正式测试"目录。当前能看到的代表性脚本包括：

- e5052b_capture_once.py：只抓相噪，不做 SPI
- case_simple_spi.py：简单 SPI case
- case_spi_then_capture.py：SPI 后立刻抓相噪
- case_spi_then_capture_no_png.py：SPI 后抓相噪但不保存截图
- case_spi_wait_then_capture.py：SPI 后等待，再抓相噪
- case_sweep_and_capture.py：边 sweep 边抓取
- case_multi_field_sweep_capture.py：多字段 sweep + 抓取
- case_preset_then_single_sweep.py：先 preset，再单 sweep
- case_discrete_sweep_capture.py：离散点 sweep + 抓取
- case_preset_then_discrete_sweep_capture.py：preset + 离散 sweep
- case_nested_discrete_sweep_capture.py：多层嵌套离散 sweep

11.1 适合怎么用

- 临时调试：优先 main.py
- 单功能验证：优先 scripts/basic/、scripts/loops/
- 正式测量：优先 scripts/fullflow/
- 新建 **case**：从 scripts/templates/ 开始改

---

12. 输出文件说明

工程已经建立了比较规整的输出目录结构。

12.1 E5052B 单次抓取输出

```
outputs/
  e5052b/
    capture_once/
      run_YYYYMMDD_HHMMSS/
        run_meta.json
        screenshots/
        traces/
        summary/
        merged/
```

12.2 Full-flow 输出

```
outputs/
  fullflow/
    case_xxx/
      run_YYYYMMDD_HHMMSS/
        run_meta.json
        screenshots/
        traces/
        summary/
        merged/
```

12.3 各类文件含义

- run_meta.json：本次运行的元信息
- screenshots/：仪器截图
- traces/：原始 CSV 数据
- summary/：单次汇总 Excel
- merged/：多步结果合并后的汇总

这套结构对后续做批量结果整理、自动归档和 GitHub 管理都很友好。

---

13. 典型使用场景

场景 1：只想确认 SPI 通不通

直接用：

```
python main.py write-read --reg 0x20 --value 0x05
```

或者：

python main.py field-write-read --name 某字段名 --value 某值

场景 2：想遍历某个字段全部取值

python main.py field-sweep --name CFG_OSC2_GMP_NUM --start 0 --stop 7 --step 1

场景 3：想改参数后立刻抓相噪

python -m scripts.fullflow.case_spi_then_capture

场景 4：想做复杂参数组合扫描

优先看：

- case_multi_field_sweep_capture.py
- case_nested_discrete_sweep_capture.py
- case_preset_then_discrete_sweep_capture.py

---

14. 新使用者最容易改的地方

通常一个新使用者接手后，最可能需要修改的是下面这些内容：

14.1 SPI 参数

改这里：

- config/active/spi_config.json

适用于：

- 改频率
- 改 CPOL/CPHA
- 改设备编号
- 改 shadow 策略

14.2 寄存器表和字段表

改这里：

- config/active/registers.json
- config/active/register_fields.json

适用于：

- 芯片版本更新
- 寄存器定义修订
- 新增字段 alias

14.3 仪器 IP 和导出行为

改这里：

- config/instrument/e5052b_config.json

适用于：

- 仪器地址变化
- 是否保存 png/csv/excel
- 抓图前等待时间

14.4 新建测试 case

建议从这里复制：

- scripts/templates/

不要一开始就在复杂 case 上直接硬改，这样更容易保持结构统一。

---

15. 工程设计上的优点

从当前代码结构看，这个项目有几个明显优点：

15.1 分层清晰

- 驱动层
- 协议层
- 配置层
- case 层
- 仪器层
- 输出层

职责划分比较合理，后续维护成本相对较低。

15.2 SPI 与仪器解耦

SPI 控制与 E5052B 抓取并没有硬绑死，这非常利于：

- 单独调 SPI
- 单独调仪器
- 在不同 case 中按需组合

15.3 已经具备“框架化”雏形

scripts/templates/、core/、utils/ 的存在，说明这个工程已经具备继续演化为稳定测试平台的基础，而不只是一次性脚本集合。

---

16. 已知注意事项

16.1 请从工程根目录运行

很多路径使用的是相对路径。如果不在根目录运行，容易出现：

- 找不到 config
- 找不到 DLL
- 输出目录生成位置不对

16.2 Windows 环境依赖 DLL

由于底层是 Ginkgo DLL，因此实际硬件联调大概率需要在 Windows 环境执行。

16.3 字段名可能存在别名与歧义

RegisterMap.resolve() 支持：

- 全名匹配
- alias 匹配
- 模糊匹配

但如果模糊匹配命中多个字段，会报歧义错误。因此正式使用时，建议尽量使用完整字段名。

16.4 outputs/fullflow/ 容易累积大量结果

长期使用后，这个目录会快速变大。建议定期清理各 case 子目录中的历史 run 内容，保留目录结构即可。

---

17. 推荐的交接使用流程

如果这是给别人接手使用，我建议对方按以下节奏上手：

第 1 天

- 阅读 README.md
- 看 main.py

- 跑 field-list
- 跑 field-read
- 跑 field-write-read

第 2 天

- 阅读 protocol/chip_spi.py
- 阅读 config/active/
- 跑 field-sweep
- 跑 scripts/basic/ 和 scripts/loops/

第 3 天

- 阅读 scripts/fullflow/
- 跑 e5052b_capture_once
- 跑 case_spi_then_capture
- 检查 outputs/ 的结果结构

这样基本就能快速独立使用这套工程。

---

18. 一句话总结

这套工程的核心思路可以概括为：

> 用 **main.py** 完成快速寄存器/字段调试，用 **scripts/fullflow/** 完成正式自动化测试，用 **config/** 控制芯片与仪器配置，用 **outputs/** 统一收集结果。

对于新使用者来说，最重要的不是一开始把所有代码看完，而是先掌握：

- 去哪里改配置
- 用什么命令做最基础验证
- 用哪个 full-flow case 跑正式测试
- 输出结果保存在哪里

掌握这四点，就已经能比较高效地上手这套自动化测试程序。